

El lenguaje java y su entorno

M4: Fundamentos de programación en java

| **AE1:** Analizar las características fundamentales del lenguaje java para el desarrollo de aplicaciones empresariales.

Introducción

¡Te damos la bienvenida a la lección- El lenguaje Java y su entorno !🎯

Java es uno de los lenguajes de programación más utilizados en la industria del software en la actualidad. Su popularidad se debe a su versatilidad y amplio uso en el desarrollo de diversos productos.

Aprender a programar en Java te brindará habilidades valiosas para crear y mantener software. Este lenguaje **es esencial para implementar nuevas funcionalidades y solucionar problemas en aplicaciones existentes.** Al dominar Java, estarás preparado para satisfacer la constante demanda de desarrolladores con conocimientos en este lenguaje.

Aprendizaje esperado

Cuando finalices la lección serás capaz de:

- Reconocer las características fundamentales del lenguaje Java para el desarrollo de aplicaciones empresariales.

¿Qué es Java?

Es un lenguaje de programación de alto nivel y orientado a objetos, que fue desarrollado por Sun Microsystems (actualmente adquirido por Oracle) en la década de 1990. Es conocido por su portabilidad, lo que significa que los programas escritos en Java pueden ejecutarse en diferentes plataformas sin necesidad de realizar cambios significativos en el código fuente.

Características del lenguaje

Java posee algunas características distintivas:

- **Portabilidad:** es altamente portátil, lo que significa que el código Java puede ejecutarse en diferentes plataformas sin cambios significativos. Esto se logra mediante el uso de la Máquina Virtual de Java (JVM), que interpreta el código Java y la ejecuta en cualquier dispositivo o sistema operativo que tenga una implementación de JVM.
- **Orientación a objetos:** está diseñado siguiendo el paradigma de programación orientada a objetos (POO). Esto significa que se basa en la creación y manipulación de objetos, lo que promueve la modularidad, reutilización de código y facilita el mantenimiento y la escalabilidad de las aplicaciones.
- **Seguridad:** tiene un enfoque fuerte en seguridad. El código se ejecuta en un entorno controlado, lo que ayuda a prevenir posibles vulnerabilidades y protege los sistemas de ejecutar código malicioso.
- **Multiplataforma:** permite desarrollar aplicaciones que son independientes de la plataforma, lo que significa que pueden ejecutarse en diferentes sistemas operativos, como Windows, macOS, Linux, entre otros.
- **Bibliotecas y comunidad amplias:** cuenta con una amplia gama de bibliotecas y frameworks disponibles que facilitan el desarrollo de aplicaciones.

¿Qué es la JVM (Java Virtual Machine)?

JVM, en español **Máquina Virtual de Java** es la encargada de interpretar y ejecutar el código Java. Proporciona portabilidad, gestión automática de la memoria y características de seguridad para las aplicaciones Java, permitiendo que se ejecuten en diferentes sistemas operativos sin necesidad de modificar el código fuente.

JDK (Java Development Kit)

Es un conjunto de herramientas que los desarrolladores utilizan para crear aplicaciones Java. Incluye el compilador de Java, que convierte el código fuente en bytecode, así como otras utilidades y bibliotecas que son necesarias para el desarrollo de software en Java.

JRE (Java Runtime Environment)

Es el entorno de ejecución de Java. Proporciona las bibliotecas y componentes necesarios para ejecutar aplicaciones Java en un sistema. Incluye la JVM, que interpreta y ejecuta el bytecode generado por el compilador.

Bytecode

Es un código intermedio que se genera cuando se compila un programa Java. Es un conjunto de instrucciones de bajo nivel que está diseñado para ser ejecutado por la JVM.

El bytecode es independiente de la plataforma, lo que significa que puede ser ejecutado en cualquier sistema operativo que tenga una implementación de JVM.

En resumen, el **JDK** es un conjunto de herramientas para desarrollar aplicaciones Java, el **JRE** es el entorno de ejecución para ejecutar aplicaciones Java y el **bytecode** es el código intermedio generado por el compilador de Java que es interpretado por la **JVM**.

¿Qué se puede hacer con Java?

Java es un lenguaje versátil y de uso gratuito que crea software localizado y distribuido.

Algunos usos comunes de Java incluyen:

- **Desarrollo de videojuegos** para móviles y computadoras.
- Java a menudo se conoce como **WORA: escribir una vez y ejecutar en cualquier lugar** (por sus siglas en inglés “Write Once and Run Anywhere”), lo que lo hace perfecto para aplicaciones descentralizadas basadas en la nube.
- Se usa para motores de **procesamiento de datos** que pueden trabajar con conjuntos de datos complejos y cantidades masivas de datos en tiempo real.
- Es una fuente inagotable de bibliotecas de machine learning. Su estabilidad y velocidad lo hacen perfecto para el **desarrollo de aplicaciones de inteligencia artificial** como el procesamiento del lenguaje natural y el aprendizaje profundo.
- **Internet de las cosas (IoT)**: Java se utiliza para programar sensores y hardware en dispositivos de periferia que pueden conectarse de forma independiente a Internet.

Evolución del lenguaje

Java ya tiene una larga historia a sus espaldas desde su primera aparición estable en 1995. En la actualidad la última versión estable es su versión 25 y cómo es de esperar su evolución no solo ha sido constante sino también, en ocasiones, revolucionaria:

- **Versión 1.0:** Introducción del lenguaje con las clases principales, la máquina virtual y el API gráfico de AWT.
- **Versión 1.1:** Inclusión de clases adicionales, estándar de JavaBeans y API de JDBC para conexión a bases de datos.

- **Versión 1.2:** Incorporación del framework de Collections y el API de Swing para interfaces de ventanas más complejas.
- **Versión 1.3:** Avances en la arquitectura de la máquina virtual con la introducción de la máquina HotSpot con compilación JIT.
- **Versión 1.4:** Fuerte soporte de XML, expresiones regulares, criptografía, entre otros.
- **Versión 1.5:** Inclusión de tipos Genéricos, metadatos con el uso de anotaciones y ampliación de APIs para programación concurrente.
- **Versión 1.6:** Introducción de un API de compilación "on-the-fly" para gestionar servicios web.
- **Versión 1.7:** Mejora de la máquina virtual con nuevos recolectores de basura.
- **Versión 1.8:** Apertura a la programación funcional con el uso de expresiones Lambda y Streams, revisión de APIs y actualización de la gestión de fechas.
- **Versión 11:** El cambio fundamental en Java 11 es sin duda JavaFX, que ha sido eliminado de la implementación estándar de la tecnología para convertirse en un módulo independiente. También coincide con un importante cambio de política de uso por parte de Oracle y el comienzo de un soporte extendido a usuarios premium.

A partir de allí, Java continuó evolucionando con un ritmo semestral de actualizaciones que introducen mejoras de rendimiento, nuevas sintaxis y funcionalidades avanzadas:

- **Versión 12:** Introducción de switch expressions en modo experimental, facilitando un código más conciso.
- **Versión 13:** Agrega text blocks (bloques de texto multilínea) y mejoras en la gestión de memoria.

- **Versión 14:** Presenta *records* y *pattern matching* para simplificar la creación de clases de datos.
- **Versión 15:** Incorpora *sealed classes* y estabiliza *text blocks* como característica oficial.
- **Versión 16:** Introduce el *Vector API* y soporte mejorado para el procesamiento paralelo.
- **Versión 17 (LTS):** Versión de soporte extendido, con mejoras en el recolector de basura y optimizaciones del rendimiento.
- **Versión 18:** UTF-8 se establece como codificación por defecto y se mejora el API de servidor HTTP.
- **Versión 19:** Introduce *virtual threads* (Project Loom) y *record patterns* en vista previa.
- **Versión 20:** Extiende las pruebas de *virtual threads* y *structured concurrency*.
- **Versión 21 (LTS):** Consolida *virtual threads*, *pattern matching for switch* y *record patterns*.
- **Versión 22:** Introduce *unnamed variables and patterns* y nuevas características en el compilador.
- **Versión 23:** Mejora el rendimiento general con nuevas optimizaciones en el recolector ZGC.
- **Versión 24:** Integra el *Foreign Function & Memory API* (Project Panama) de forma estable.
- **Versión 25:** Continúa la integración de *Project Valhalla* (tipos de valor) y optimiza la ejecución de *virtual threads*.

Estas versiones representan importantes hitos en la evolución de Java, incorporando nuevas funcionalidades, mejoras en el rendimiento y adaptaciones al paradigma moderno de desarrollo.

Estas versiones representan importantes hitos en la evolución de Java, incorporando nuevas funcionalidades y mejoras en el lenguaje y en las bibliotecas disponibles

Descarga e instalación del JDK en Windows

La escritura de aplicaciones y applets de Java requiere herramientas de desarrollo como el **JDK (Java Development Kit)**.

El JDK incluye el **Java Runtime Environment (JRE)**, el **compilador javac**, y un conjunto completo de **APIs y utilidades** necesarias para compilar, ejecutar y depurar programas Java.

Su instalación es sencilla tanto para programadores nuevos como para desarrolladores con experiencia.

A continuación, se muestra el paso a paso para descargar e instalar **Java SE Development Kit 25**:

1. **Ir al sitio oficial de Oracle Java SE**

Accede a:

<https://www.oracle.com/java/technologies/downloads/>

2. **Seleccionar la versión adecuada**

Desplázate hacia abajo hasta la sección **“Java SE Development Kit 25”**.

Allí verás los instaladores disponibles para los diferentes sistemas operativos (**Windows, macOS y Linux**).

3. **Aceptar los términos de licencia**

Marca la opción para **aceptar la licencia de Oracle** antes de iniciar la descarga.

4. **Elegir el instalador correcto para Windows**

Para sistemas Windows modernos (64 bits), selecciona el archivo con extensión **.msi** (instalador automático) o **.zip** (instalación manual).

5. **Descargar el archivo de instalación**

Guarda el archivo en una carpeta de tu elección, por ejemplo en **“Descargas”**.

6. **Ejecutar el instalador**

Haz doble clic sobre el archivo **.msi** descargado.

El asistente de instalación te guiará paso a paso para completar la instalación del **JDK 25**.

7. Verificar la instalación

Abre la terminal o símbolo del sistema (**CMD**) y escribe:

```
java --version
```

Si la instalación fue correcta, deberías ver un mensaje similar a:

```
java 25 2025-09-16 LTS
Java(TM) SE Runtime Environment (build 25+37-LTS-3491)
Java HotSpot(TM) 64-Bit Server VM (build 25+37-LTS-3491, mixed mode, sharing)
```

¿Qué es un IDE?

IDE es la abreviatura en inglés de **Entorno de Desarrollo Integrado**. Es un sistema de software para el diseño de aplicaciones, que brinda varias herramientas comunes para desarrolladores en una sola interfaz gráfica. Combina un editor de texto avanzado con características como resaltado de sintaxis, autocompletado de código y sugerencias, facilitando la escritura de código.

Otras características que es común encontrar en un IDE son: navegador del código fuente, buscador, integración con sistemas de control de versión como Git, asistentes para reestructurar (refactor), indicador de errores y advertencias de malas prácticas y ejecutar pruebas (tests).

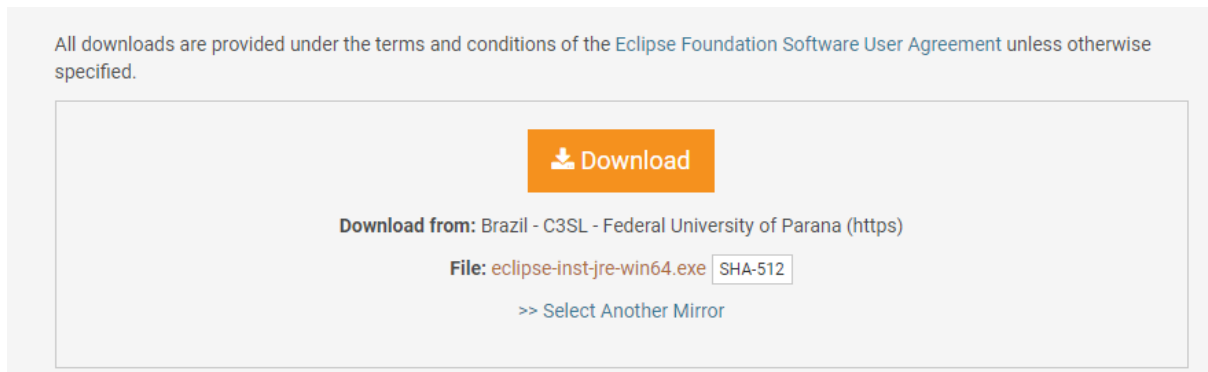
Eclipse IDE

Eclipse es el IDE más usado para el desarrollo en Java. Es de código abierto y gratuito. Permite extender sus funciones mediante el desarrollo de plugins. Existen muchas versiones de Eclipse para programar en diferentes lenguajes e incluso posee varias versiones para Java.

Descarga e instalación del IDE en Windows

1. Ve al sitio web oficial de Eclipse en: <https://www.eclipse.org/downloads/>

En esa página encontrarás el botón de descarga, al hacerle clic te llevará a la siguiente pantalla:

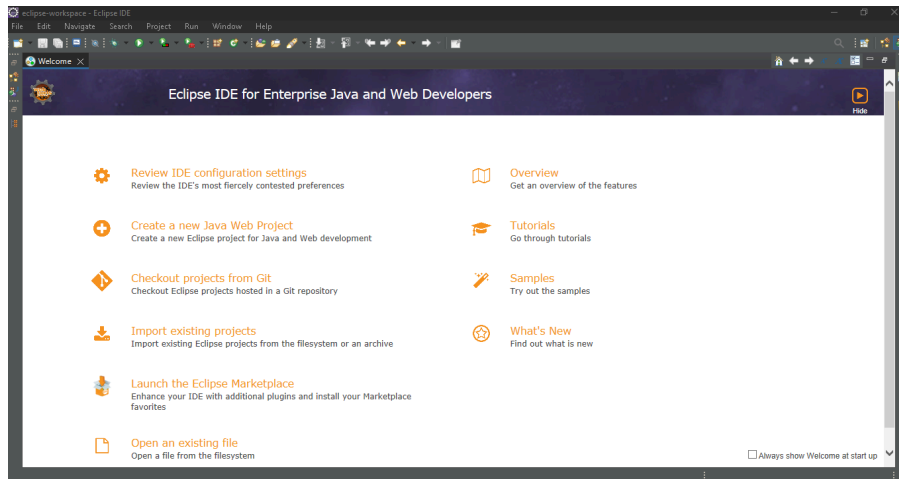


2. Una vez que se haya completado la descarga, abre el archivo de instalación. Verás que se despliegan varias opciones de IDE para seleccionar. Para este caso, recomendamos descargar la versión **Eclipse IDE for Enterprise Java and Web Developers**
3. Sigue las instrucciones del asistente de instalación para configurar Eclipse. Puedes elegir la ubicación de instalación y las opciones adicionales según tus preferencias.
4. Una vez que la instalación haya finalizado, inicia Eclipse. Te pedirá que selecciones un "Workspace" (espacio de trabajo), que es la ubicación donde se almacenarán tus proyectos.
5. Después de seleccionar el "Workspace", aparecerá la ventana principal de Eclipse. ¡Ahora estás listo para empezar a desarrollar!

Recuerda que Eclipse es altamente personalizable y puedes instalar complementos y ajustar la configuración según tus necesidades.

Creación de un programa en Java

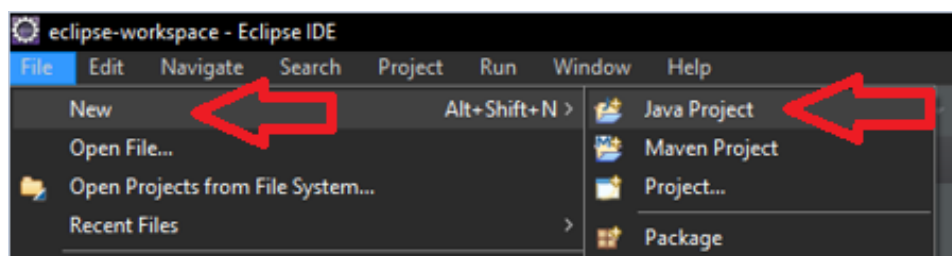
Vamos a crear un programa desde cero. Para esto empezaremos por abrir el Eclipse IDE, donde verás la pantalla de bienvenida:



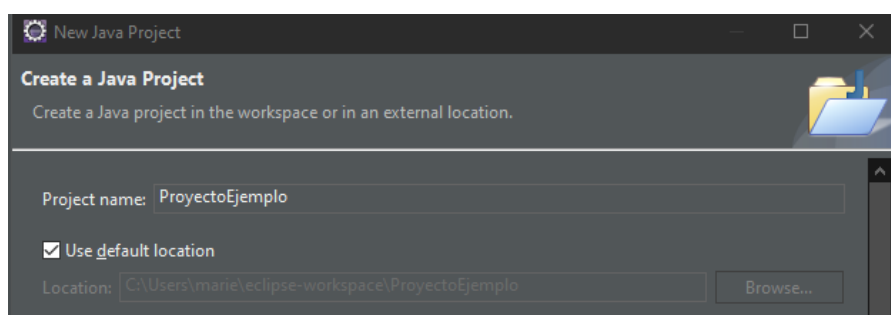
Esta pantalla ofrece información sobre el IDE y posee algunos atajos para poder acceder a diversas funcionalidades. Puedes cerrar la pestaña desde la parte superior izquierda.

Luego, dirígete hacia el menú superior y selecciona la primera opción: **File**.

Al hacer clic, se desplegarán las opciones en donde debes seleccionar **New**, y a continuación **Java Project**.



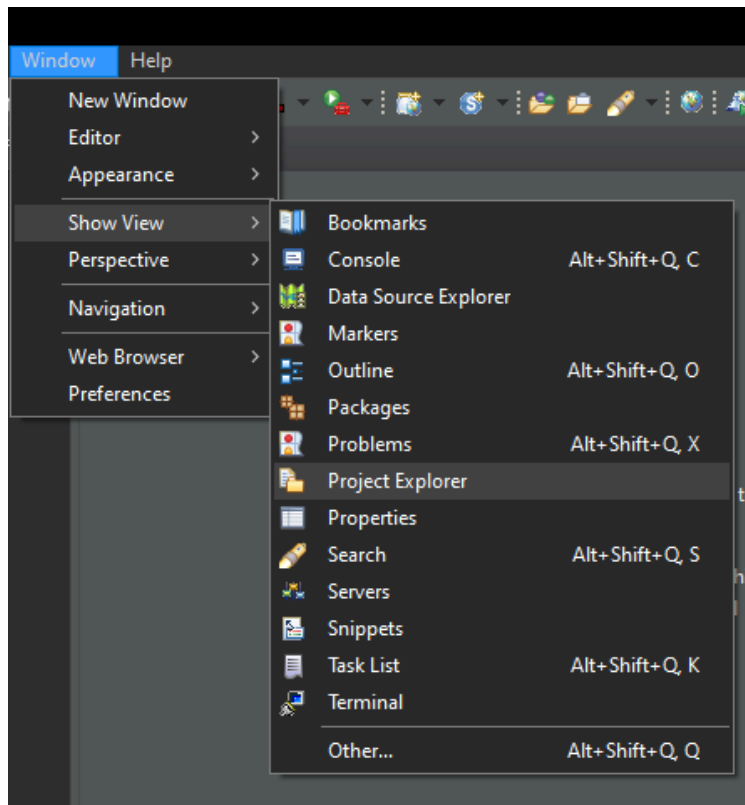
A continuación se abrirá una nueva ventana con el nombre **New Java Project**, en dónde inicialmente sólo colocaremos el nombre de nuestro proyecto. Para este ejemplo, utilizaremos el nombre **ProyectoEjemplo**.



Al final de la ventana, encontraremos el botón **Next >** donde haremos clic.

La siguiente ventana nos muestra las configuraciones por defecto del proyecto que estamos generando. Por ahora no realizaremos más cambios y por último presionamos **Finish**.

Ahora volvimos a la ventana principal, donde deberías ver el explorador de proyectos del lado izquierdo. De no ser así, se habilita haciendo clic en la barra de menú, en la pestaña **Window**, opción **Show View**, y por último **Project Explorer**. De esta manera tendrás en la vista todo el esqueleto de tu primer proyecto Java.



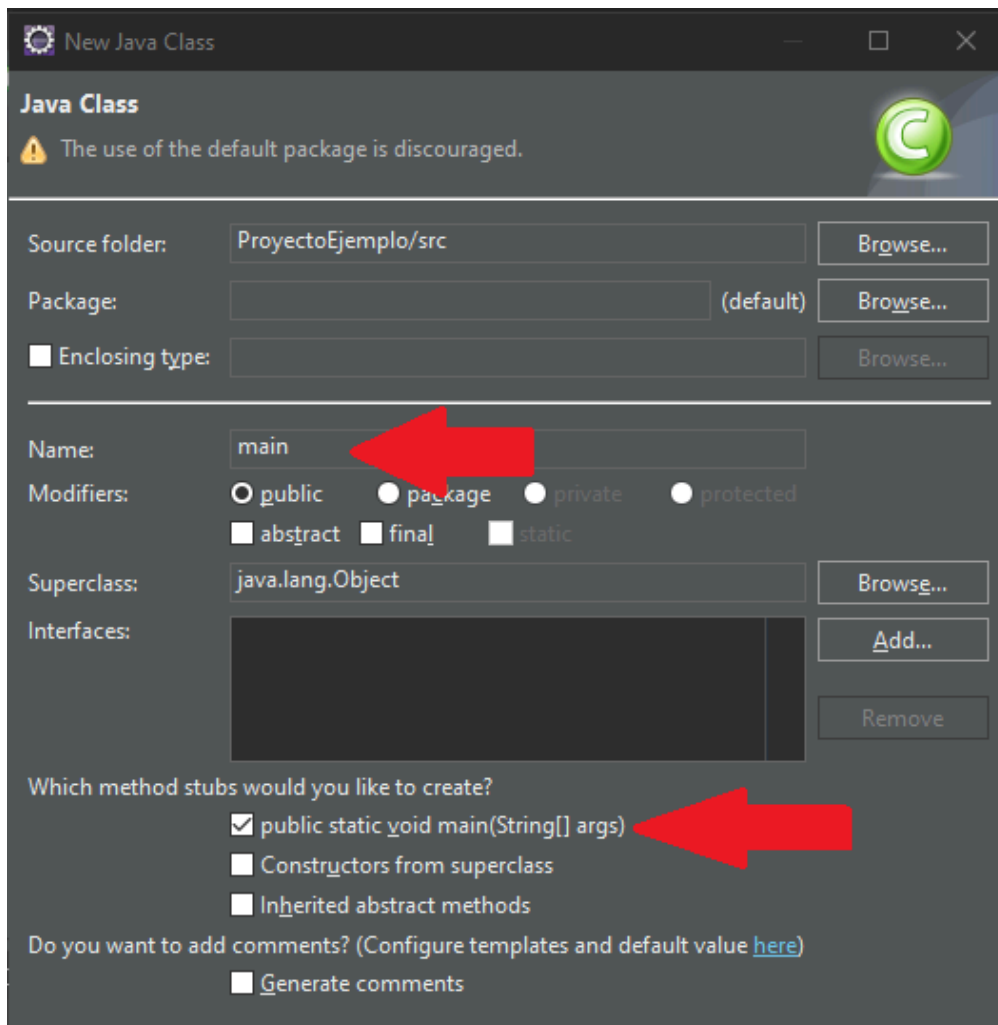
Ahora que puedes ver el esqueleto de tu proyecto del lado izquierdo, vamos a hacer clic en la carpeta “**src**”, donde haremos clic derecho para seleccionar la opción **New**, y luego la opción **Class**.

Se abrirá una nueva ventana de configuración, donde colocaremos el nombre de nuestro primer archivo ejecutable.

Para esto, primero debemos ponerle un nombre al archivo Class, en nuestro caso le colocaremos de nombre “**main**”.

Utilizamos la palabra main porque es el archivo donde estará el **método Main**, que es el punto de entrada de un programa ejecutable; es donde se inicia y finaliza el control del programa. Para que este método se encuentre en nuestra

Class, vamos a tildar la opción **public static void main(String[] args)**, y terminamos presionando Finish.



Ya tenemos nuestra primera clase ejecutable para comenzar a escribir código Java. Este debe escribirse entre las llaves de apertura y cierre, donde podemos ver el texto **//TODO Auto-generated method stub**.

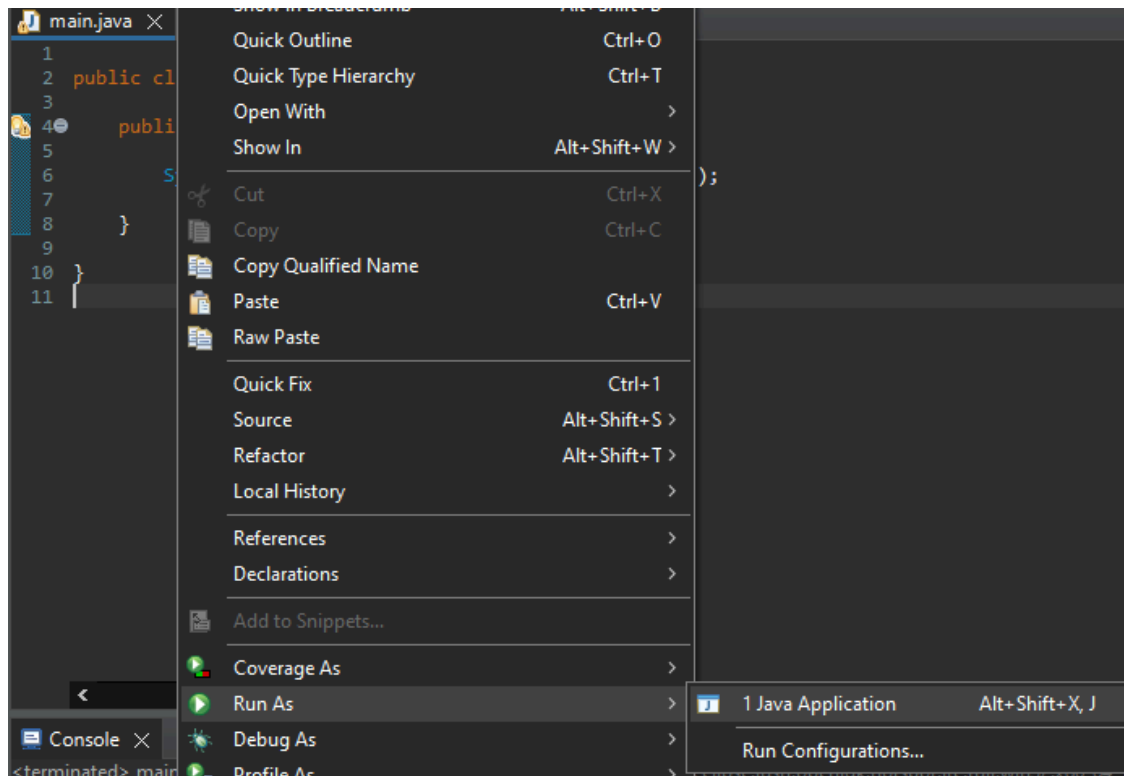
Este renglón se encuentra en color gris porque es un **“comentario”**. Los comentarios sirven para introducir líneas de texto o código de referencia que **no serán ejecutados** por el compilador. También se utiliza para escribir descripciones o documentar las funcionalidades de cada método.

En Java, los comentarios comienzan con dos barras seguidas (//).

```
public class main {
    public static void main(String[] args) {
        // TODO Auto-generated method stub
    }
}
```

Para ejecutar un programa en Java, debemos hacer **clic derecho** en cualquier parte del editor y seleccionar **Run As**, y luego **Java Application**.

También puedes presionar el botón con flecha verde que se encuentra en la parte superior del IDE, donde está la barra de herramientas.



Cierre 🖊️

Java es un lenguaje de programación ampliamente utilizado en la industria del software. La evolución de Java a lo largo de sus versiones ha introducido nuevas características y mejoras significativas.

El JDK es un conjunto de herramientas para el desarrollo de aplicaciones Java, mientras que el JRE proporciona el entorno de ejecución necesario para ejecutar aplicaciones Java.

Eclipse IDE es un entorno de desarrollo integrado que facilita la escritura, compilación y depuración de código Java, ofreciendo herramientas y características adicionales para agilizar el proceso de desarrollo.

Con Eclipse IDE y las herramientas proporcionadas por el JDK, los desarrolladores pueden crear y ejecutar aplicaciones Java de manera eficiente y productiva.

¡Nos vemos de nuevo, más adelante! 🙌

Referencias

- OpenWebinars. (s.f.). Qué es Java: Principios básicos y evolución. <https://openwebinars.net/blog/que-es-java-principios-basicos-y-evolucion/>
- Amazon Web Services. (s.f.). What is Java? Recuperado de <https://aws.amazon.com/what-is/java/>
- The Eclipse Foundation. (s.f.). Download Eclipse — Technology that is right for you. Recuperado de <https://eclipse.org/downloads/>
- Javatpoint. (s.f.). How to use Eclipse for Java. Recuperado de <https://www.javatpoint.com/how-to-use-eclipse-for-java-beginners/>

¡Muchas gracias!

Nos vemos en la próxima lección

